

RestOps — Full Frontend Click-Through Walkthrough

Before you start

Run both servers: backend (`node server.js` or however you normally start it, with MongoDB running as a replica set — remember `mongod --replSet rs0`) and frontend (`npm run dev` in the `frontend` folder).

Open your browser's DevTools (F12) and keep the **Console** and **Network** tabs visible as you click through — this is how you'll catch silent failures. A red error in the console or a failed (red) request in Network is a bug, even if the page visually looks fine.

For anything socket-related, you'll need **two browser windows side by side** (or one normal + one incognito) logged in as two different roles at once — e.g. a Host in one window and a Customer in another — so you can watch live updates actually happen in real time. These are called out specifically below. Keep a running list as you go: for each *X* (failed) item, write down the page, what you clicked, what you expected, what actually happened, and any console error text. That list becomes your bug backlog.

Legend: **Action** = what you do. **Expect** = what should happen. **Red flag** = a sign something's wrong even if it's not an obvious crash.

PART 1 — Auth & Role Redirect

1.1 Register a fresh customer

Action: Go to `/register`, fill in name/email/password, submit. **Expect:** Redirected to Discovery/Customer Home, logged in automatically (or to `/login` if auto-login wasn't built — either is fine, just confirm which happens). **Red flag:** Landing on a blank page, or an unhandled error in console about `undefined` properties (usually means the response shape didn't match what the frontend expected).

1.2 Log out, log back in as that customer

Action: Use the Logout button (wherever it lives — NavBar or Account page), then log back in with the same credentials at `/login`. **Expect:** Redirected to Discovery/Home again. **Red flag:** Being sent to a role-dashboard that doesn't match — a customer should never land on `/manager` etc.

1.3 Create a manager account and confirm redirect

Action: Since a "manager" role only exists after being assigned (per the owner flow), skip ahead to Part 3 first to create one, then come back and log in as that manager directly. **Expect:** Login redirects straight to `/manager`, not `/discovery`.

1.4 Repeat for waiter, chef, cashier, host

Action: Once you've created these accounts via Manager's Staff page (Part 4), log in as each one separately. **Expect:** Each lands on its own dashboard — `/waiter`, `/chef`, `/cashier`, `/host` respectively. **Red**

flag: Any role landing on the wrong dashboard, or a 403/blank page.

1.5 Protected route check

Action: While logged OUT (clear localStorage or use an incognito window), try to directly navigate to `/manager`, `/reservations/mine`, `/owner`. **Expect:** Redirected to `/login` for all of them. **Red flag:** Any of these pages rendering content without being logged in.

PART 2 — Customer Journey

2.1 Discovery

Action: Go to `/restaurants` (or `/discovery`). Try the search box, try clicking a cuisine filter chip. **Expect:** Restaurant cards update as you filter/search. Clicking a cuisine chip again deselects it. **Red flag:** Filters doing nothing, or the list not updating at all (check Network tab — is the request even firing with the right query param?).

2.2 Restaurant Detail

Action: Click a restaurant card. **Expect:** Lands on `/restaurants/:id` showing name/address/cuisine, a menu preview (first few items), and a "Book a Table" button. If not logged in, menu preview should show a graceful "log in to view menu" message, not a broken request. **Red flag:** Menu preview silently empty with no explanation when logged out (should show the fallback message, per how this was built).

2.3 Full Menu View

Action: Click "View Full Menu." **Expect:** Categorized list of items, only available ones shown (if you toggled any off as a manager earlier, confirm they're actually missing here). **Red flag:** Unavailable items showing up — this would mean the availability filter isn't applied client or server-side.

2.4 Booking Step 1 (Party Size & Time)

Action: Click "Book a Table," fill in date/time/party size. **Expect:** Can't submit with a past date or invalid party size — validation should block it with a visible message. Valid input enables "Continue." **Red flag:** Being able to submit a booking for a date in the past.

2.5 Table Selection

Action: Continue to the table/combination selection page. **Expect:** A floor grid renders (if the manager built one for this restaurant — if not, expect an empty/graceful state, not a crash). If your party size fits one table, you should see a direct table option. If it needs combining, you should see ranked suggestion CARDS instead of being able to freehand-select multiple tables yourself. **Red flag:** Being able to manually multi-select tables when a combination should route through suggestions instead. Also check: does clicking a suggestion actually highlight the right tables on the grid?

2.6 Booking Confirmation

Action: Confirm your table choice. **Expect:** Lands on the confirmation page with a visible countdown

timer ticking down from the lock duration. If your booking needed approval, you should see a "waiting for approval" state instead of a Confirm button. **Red flag:** No countdown moving at all (frozen timer = a bug in the timer logic, not just a visual nit).

→ **SOCKET TEST (two windows):** With the customer's booking still waiting for approval in Window A, go to Window B logged in as the Manager, approve it from the Approvals page. **Expect:** Window A should transition out of the waiting state on its own within a few seconds, without you refreshing it. **Red flag:** Window A stays stuck waiting even after approval — means the socket listener (or the events we added, `reservation:approved`) isn't wired correctly.

2.7 Confirm the booking

Action: Click Confirm (once available). **Expect:** Redirected to My Reservations, and the new reservation appears there as "confirmed" or "locked" (whichever your flow ended on). **Red flag:** Reservation missing from the list, or stuck showing an old status.

2.8 My Reservations

Action: View the list. Try cancelling an eligible one. **Expect:** Upcoming/Past/Cancelled sections are visually distinct. Cancel button works and the item moves to Cancelled. **Red flag:** Cancel button appearing on a reservation that's already completed/cancelled (shouldn't be clickable there).

2.9 Reservation Detail → Order Food

Action: Click into a confirmed/seated reservation, click "Order Food." **Expect:** "Order Food" button ONLY appears for confirmed/seated reservations — check it's genuinely absent on a locked or cancelled one. **Red flag:** Order button showing on a reservation that isn't actually confirmed yet.

2.10 Place an Order

Action: Add items with the quantity steppers, watch the running total update, submit. **Expect:** Order submits, page transitions into a live tracking view showing "pending."

→ **SOCKET TEST (two windows):** With this order tracking open in Window A (Customer), go to Window B as Chef, move the order through Accept → Preparing → Ready. **Expect:** Window A's status stepper should advance live at each step without a refresh. Then in Window B as Waiter, do Pickup → Deliver — Window A should reflect "served" live too. **Red flag:** Any step not updating live — tells you exactly which socket event isn't reaching the frontend.

2.11 Confirm Receipt

Action: Once status is "served," click "Confirm Received." **Expect:** Succeeds, no longer prompts again.

2.12 My Orders

Action: Go to `/orders/mine`. **Expect:** This order appears in history with correct status and total.

PART 3 — Owner Journey

3.1 Become an Owner

Action: From a logged-in customer account, find "Own a restaurant" (Landing page CTA, or the NavBar/Account link if you fixed the onboarding gap), fill in name/address/phone/cuisine, submit. **Expect:** Redirected to `/owner`, your new restaurant appears as a card, your role is now genuinely "owner" (check via Account page or console-inspecting the auth state). **Red flag:** Staying on the create form after submit, or landing on `/owner` but seeing a blank dashboard (auth context might not have refreshed with the new role).

3.2 Assign a Manager

Action: Click into that restaurant's manager assignment page. Search by email for a user (use another test account you registered earlier), assign them. **Expect:** Success, the assigned manager's name/email now shows on this page and back on the Owner Dashboard card. **Red flag:** Searching an email that doesn't exist should show a clear "not found" message, not a crash.

3.3 Replace/Remove Manager

Action: Try removing the manager, then assigning a different one. **Expect:** Both actions work cleanly, dashboard reflects the current state accurately each time.

3.4 Restaurant Overview

Action: View the read-only overview page for this restaurant. **Expect:** Floor plan, menu, and staff sections all show SOMETHING (even if empty-state, since you likely haven't built these out yet as manager). No edit controls anywhere on this page. **Red flag:** Any of the three sections crashing the whole page instead of failing gracefully on its own (this was specifically flagged as a risk when we built this page).

PART 4 — Manager Journey

(Log in as the manager account you assigned in 3.2)

4.1 Manager Dashboard

Action: Land on `/manager`. **Expect:** Snapshot cards for approvals/waitlist/kitchen counts, a mini floor plan preview. **Red flag:** Counts showing incorrect numbers or never updating.

4.2 Floor Plan Builder — the big one

Action: Go to Floor Plan Builder. Add a few tables (try different shapes/capacities), add a wall/window, click a table to edit its capacity/combinable/adjacency, link two tables as adjacent. **Expect:** Every placement/edit reflects immediately in the UI. Adjacency linking visually shows the connection somehow. **Action:** Click "Save Layout." **Expect:** Success confirmation. **Action:** Refresh the whole page (hard refresh, Ctrl+Shift+R). **Expect:** Everything you placed and edited is still there, loaded fresh from the backend — this is the critical test, since it proves the save/reload round-trip actually works, not just the in-memory UI

state. **Red flag:** Anything missing or reset after refresh — means the save didn't actually persist correctly, or the load-on-mount isn't reading it back properly.

4.3 Staff Management

Action: Add a waiter, chef, cashier, and host (you'll need these for later parts). Remove one, confirm it disappears. **Expect:** All CRUD works, role badges display correctly per person.

4.4 Menu Management

Action: Add a few menu items across categories. Toggle one's availability off. Edit one's price. Delete one. **Expect:** All actions work, and — important — go back to the Customer's Menu View for this restaurant afterward and confirm the toggled-off item is genuinely hidden there.

4.5 Reservation Approvals

Action: With a pending multi-table booking sitting in the queue (create one as a customer in another window if needed), view it here. **Expect:** Shows party size, requested tables, floor thumbnail highlighting them. Approve/Reject both work and remove it from the list.

→ **SOCKET TEST:** Create a new combo-booking as a Customer in Window B while this Approvals page is open in Window A. **Expect:** It should appear on this list live, without refreshing Window A.

4.6 All Reservations

Action: View the full list, try the status and date filters. **Expect:** Filtering actually narrows the list correctly.

4.7 Waitlist Dashboard

Action: View waitlist entries (create some via Host's Seat Walk-in in another window if the list is empty). **Expect:** Position and wait-time numbers look sensible, guest name fallback shows correctly for walk-ins with no account.

PART 5 — Host Journey

(Log in as the host account)

5.1 Host Live Floor View

Action: View the floor grid. **Expect:** Same layout the manager built, color-coded by live status. Click a table — see the contextual action popover.

→ **SOCKET TEST:** In Window B as a Customer (or via a direct backend action), change a table's status. **Expect:** Window A's floor view color updates live.

Action: Try "Mark as Cleaning" on an available table, then toggle it back. **Expect:** Status updates correctly both directions.

5.2 Seat a Walk-in

Action: Go to Seat Walk-in, enter a party size and guest name, submit. **Expect:** Either an immediate table assignment confirmation, or a "joined waitlist" state if nothing fits — try both scenarios (a small party that clearly fits, and a huge party that clearly won't) to see both paths.

5.3 Host Waitlist View

Action: Respond to a notified entry (accept or decline). **Expect:** Action happens immediately without an extra confirmation step (this was deliberately built to skip the confirm dialog for speed).

5.4 Approvals

Action: Same as Manager's version — confirm it works identically from this role.

PART 6 — Waiter Journey

(Log in as the waiter account — first have a manager assign this waiter to at least one table via the backend/staff page if that UI exists, or directly via the assign-waiter endpoint if the UI for it wasn't built into Staff Management)

6.1 My Tables

Action: View assigned tables. **Expect:** Only shows tables actually assigned to this waiter — NOT every table in the restaurant. **Red flag:** Seeing all tables regardless of assignment — means the scoping isn't working, or no assignment UI exists yet to actually test this meaningfully (in which case, note this as still-needed work, not a pass).

6.2 Take an Order

Action: Click into a table, add items, submit. **Expect:** Order created successfully, confirmation shown.

6.3 Kitchen Handoff

Action: Once an order reaches "ready" (push it through as Chef in another window), come back here.

Expect: It appears in the Ready section. Click Picked Up, then Deliver. **Red flag:** The order never appearing here at all — this is exactly the bug we caught and fixed (`GET /api/kitchen` missing 'ready' status) — if it's still missing, the fix didn't take.

PART 7 — Chef Journey

(Log in as chef)

7.1 Kitchen Board

Action: With a fresh order sitting in Pending (place one as a customer or waiter first), drag/click it through all 4 columns: Accept → Preparing → Ready. **Expect:** Each transition works, card visually moves columns, no ability to skip a stage out of order. **Red flag:** Being able to jump straight from Pending to Ready, or the card not moving after a successful action.

→ **SOCKET TEST:** With this board open, place a NEW order from a different window (Customer or Waiter).
Expect: It appears on the Pending column live, without refreshing.

7.2 Focus check

Action: Look at the page overall. **Expect:** No navigation to tables/menu/floor plan/reservations anywhere — confirm this page really is as minimal as intended.

PART 8 — Cashier Journey

(Log in as cashier)

8.1 Pending Bills

Action: With a completed (but unpaid) order existing, view this list. **Expect:** Shows table, item summary, total.

8.2 View & Pay

Action: Click to expand the itemized bill, select a payment method, Mark Paid. **Expect:** Succeeds, order disappears from the pending list.

8.3 THE CRITICAL BACKEND TIE-IN CHECK

Action: If this order involved a COMBINED table group (created via Manager/Host override earlier), after marking paid, check the floor view (as Host or Manager) for ALL tables in that group. **Expect:** Every table in the group shows as available again, not just the one directly tied to the order. **Red flag:** Any table in the group staying stuck "occupied" — this is the exact bug the `combinedGroupId` fix was built to solve; if it's still broken, that fix didn't fully take in the frontend/backend integration.

Wrapping up

After going through all 8 parts, you should have a list of X items with enough detail (page, action, expected vs actual, console errors) to turn into fix prompts — the same format we've used for every bug so far. Bring that list back and we'll work through them the same way: smallest, most isolated fix first, re-test, move to the next.

Don't try to fix things mid-walkthrough — finish the full pass first so you have the complete picture, since some bugs might actually be duplicates or share a root cause once you see the whole list together.